OWASP

Open Web Application Security Project

Payload-Driven Recon Scans And New Module Workflows

for Google Summer of Code 26’

Aarush Aarush
IIT Madras (2024-2028)
Possible project mentors:
Sam Stepanyan, Arkidii Yakovets

Project Overview

This project focuses on enhancing the scanning capabilities of Nettacker by introducing payload-based probing techniques for service detection. By sending

protocol-aware probes instead of generic requests, the scanning engine will be able to identify running services more accurately and extract version information where possible, significantly improving reconnaissance capabilities.

In addition, the project will implement and extend several improvements suggested in the idea list for Nettacker, addressing current limitations in the scanning workflow and module execution pipeline. These enhancements aim to make scans more reliable, scalable, and informative.

Overall, the proposed improvements will strengthen Nettacker's ability to perform reconnaissance scan and vulnerability discovery while expanding its coverage of vulnerabilities listed in the CISA Known Exploited Vulnerabilities (KEV) catalog.

Table of contents

Personal Information

- Name: Aarush Aarush
- University: Indian Institute of Technology, Madras
- Degree: B.tech
- Major: Computer Science and Engineering
- Residence: Madras/Chennai, India
- Time Zone: Indian Standard Time (UTC + 5:30)
- GitHub Account: GitHub
- Email:
    1. cs24b064@smail.iitm.ac.in (Github linked + Institute mail)
    2. aarush289156@gmail.com (personal mail)
- Language spoken: English
- Slack username: AARUSH

I am Aarush, a 2nd year undergraduate student pursuing a B-tech degree at the Indian Institute of Technology, Madras (IITM), which has been ranked the best engineering institute in the country for the last 5 consecutive years .

I have a lot of hobbies, which I engage in other than writing code. I love playing cricket , badminton and many other outdoor sports .

Programming background

General:

I started programming after joining the institute as I had many programming courses and learnt software development aspect of programming through the Projects under various clubs in my institute .
There are two fields that I actively take part in now, Computer Networks and Cybersecurity . I had a lot of fun and learning associated with both of these fields.

Cybersecurity experience:

I have been working on 2 cybersecurity projects under the cybersecurity club of IITM with project lead being @pUrGe12 .

1. Wifi - deauth and evil-twin attack : Under this project , I have done security analysis of WPA/WPA2 networks by implementing controlled Wi-Fi deauthentication attacks and studying 802.11 authentication and handshake behavior. Evaluated how attackers can disrupt client connectivity and potentially exploit reconnection processes.
This was the point when I first used nettacker , my project lead introduced Nettacker to me and we used the tool for our project to scan the open ports and some service detection on the target .

2. Oblivious HTTP : Studying the Oblivious HTTP (OHTTP) protocol with a focus on its threat model, relay-based anonymity, and privacy guarantees in modern web systems. Currently developing a Python library that implements request encapsulation, key handling, and relay interaction. The project is in the development phase and aims for a future open-source release.

Python experience

My major tech stacks are C++ and python. Since C++ is being used in my institute in courses , I know C++ very well and use it in competitive programming contests. I have learnt Python majorly through the club projects and contribution to Nettacker . I have used python for the club projects and the competitions conducted by many HFTs in our institute like low-latency conducted by IMC where I secured 3rd position in the institute and many other competitions .

Git and GitHub experience

I have been using git for over one and a half years now and I have become familiar with the workflows involved in git and GitHub. I have learnt about rebasing, signing commits ,resolving merge conflicts etc. by making a lot of mistakes in my PRs while contributing to OWASP. I have also learnt about the best practices in GitHub (like creating a new branch for every contribution) during my contribution to the Nettacker project . Prior to my contributions to OWASP ,

Cybersecurity always fascinated me and since the Command line interface is a helpful tool in the cybersecurity field so I made myself familiar with the command line interface and since I have been using Linux OS for last 5-6 months , I am very well versed with most of the commonly used Commands in CLI .

Platform details

Operating system (primary): Fedora Linux 42 (Workstation Edition), x86_64
Operating system (secondary): Windows 11 (dual booted)
Development setup: Visual Studio Code, GNOME Text Editor
Virtual machines: Kali Linux, REMnux (VirtualBox)
Computer hardware: Dell G15 5530, 13th Gen Intel Core i7-13650HX CPU, 16GB RAM, NVIDIA RTX 4060 GPU

Motivation behind participating in GSoC 26’

My primary motivation behind applying to OWASP is based on my interest in cybersecurity. I have followed OWASP’s top 10 since 2024 (started just before joining the college when I was exploring the field of cybersecurity)
I have used Nettacker for my projects and I enjoyed working with this tool because it matched my tech stack and hence, I would like to add more features and support to this tool so as to make it more useful for all the users and make this tool more widely adopted .

I had realised that there are some things that would improve Nettacker, that I can help develop. These include:

- Module flow execution
- Service logging functionality
- Version scanning functionality
- TCP/UDP service detection

I tried to implement some of them through my PRs (will be mentioned below) but there remains a lot to be done. I wish to utilize the opportunity that GSoC presents to bring these ideas to life in Nettacker while also working on the ones **mentioned by the organisation**.

Over the past few months, my journey into open source has given me much more than just technical exposure—it has reshaped how I view collaborative development. What stands out to me is the passion with which people across the world build and maintain projects, often driven purely by curiosity, dedication, and the desire to create tools that others can freely benefit from.

Witnessing this spirit of collaboration has been incredibly motivating, and becoming an active contributor to such a community is something I strongly aspire to do.

Contributions to Nettacker

I started contributing to the OWASP community and Nettacker in February 2025. Below is a list of my contributions that are relevant to my current project.

Merged

- Nettacker PR#1162: Added the SMB service fingerprints in the port scan to prevent the crash in the modules execution
- Nettacker PR#1242: Fixed the header key in multiple modules
- Nettacker PR#1268: Added a new CISA KEV Module
- Nettacker PR#1270: Documented all the modules to prevent duplicate modules PRs
- Nettacker PR#1277: Fixed the status field in a module to prevent false positives
- Nettacker PR#1286: Fixed regex field of a module
- Nettacker PR#1365: Added a new CISA KEV module
- Nettacker PR#1319 - Enhanced the test file (test_yaml_regexes) to do a comprehensive schema check as well along with regex validation for modules

Open

- Nettacker PR#1163 - Validate the targets(resolvable or not) before executing the modules to improve scan efficiency

- Nettacker PR#1225 - Added new CISA KEV module for CVE_2025_8848
- Nettacker PR#1227 - Added new CISA KEV module for CVE_2025_53558
- Nettacker PR#1228 - Added new CISA KEV module for CVE_2025_55169
- Nettacker PR#1233 - Added new CISA KEV module for CVE_2025_54589
- Nettacker PR#1235 - Added new CISA KEV module for CVE_2025_2776
- Nettacker PR#1237 - Added new CISA KEV module for CVE_2025_12480
- Nettacker PR#1238 - Added new CISA KEV module for CVE_2025_58360
- Nettacker PR#1240 - Added new CISA KEV module for CVE_2025_64446 (under draft, dependent on dynamic random string generator)
- Nettacker PR#1243 - Added new CISA KEV module for CVE_2025_11371
- Nettacker PR#1282 - Enhance WAF detection module with updated signatures and payloads
- Nettacker PR#1287 - Add dynamic random string generator for modules
- Nettacker PR#1314 - Added new CISA KEV module for CVE_2026_25892

Issues opened

- Nettacker #1241 - Silent false positives due to wrong header field in many modules
- Nettacker #1269 - Incomplete documentation of modules
- Nettacker #1276 - Wrong status field in a module
- Nettacker #1280 - HTTP modules silently failed due to field “body” instead of “data” in modules
- Nettacker #1285 - Module wp_plugin crashing because of wrong regex in status field
- Nettacker #1312 - Redundant success_conditions field in wp_plugin module
- Nettacker #1318 - test_yaml_regexes (test file) validates only the regexes and not the basic structure of modules (schema validation)

Proposal

Following from the discussions in #project-nettacker, my talks with @securestep9 and the idea list mentioned in OWASP's official site, below are my proposed improvements for Nettacker **arranged** according to relative priority.

- **Improving host scanning**
  High priority
  - Support for Module Work-flows - Code for reference: @f0df79b and @a5583c3
  - Improvement in service detection , version detection and payload-based probes - Code for reference: changelog

- **External API key configuration**
  - Support for external API keys - Code for reference: changelog

- **Extractors integration**
  - Support for extractors in modules - Code for reference: changelog

- **Web improvements** - Code for reference: changelog

  I will also work on implementing new CISA KEV modules , maximizing Test coverage and a detailed outlook on that is presented in the timeline.

Improving Module Work-flows

1. Stop-at-first-match

- In the current implementation of modules , it creates many substeps out of each step which then logs the success to the database , independent of other sub-steps from the same step , due to which there are multiple entries in the final graph table .
  An example for this is waf_scan module which creates a lot of duplicate entries :-

2026-03-17 19:02:19.287424	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:19.614687	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:20.070115	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:20.133597	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:20.967356	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:20.671002	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:22.565314	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:21.697074	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:23.270788	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:23.188918	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:22.772920	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found
2026-03-17 19:02:23.483397	cloudfare.com	waf_scan	443	Cloudflare (Cloudflare Inc.) Found

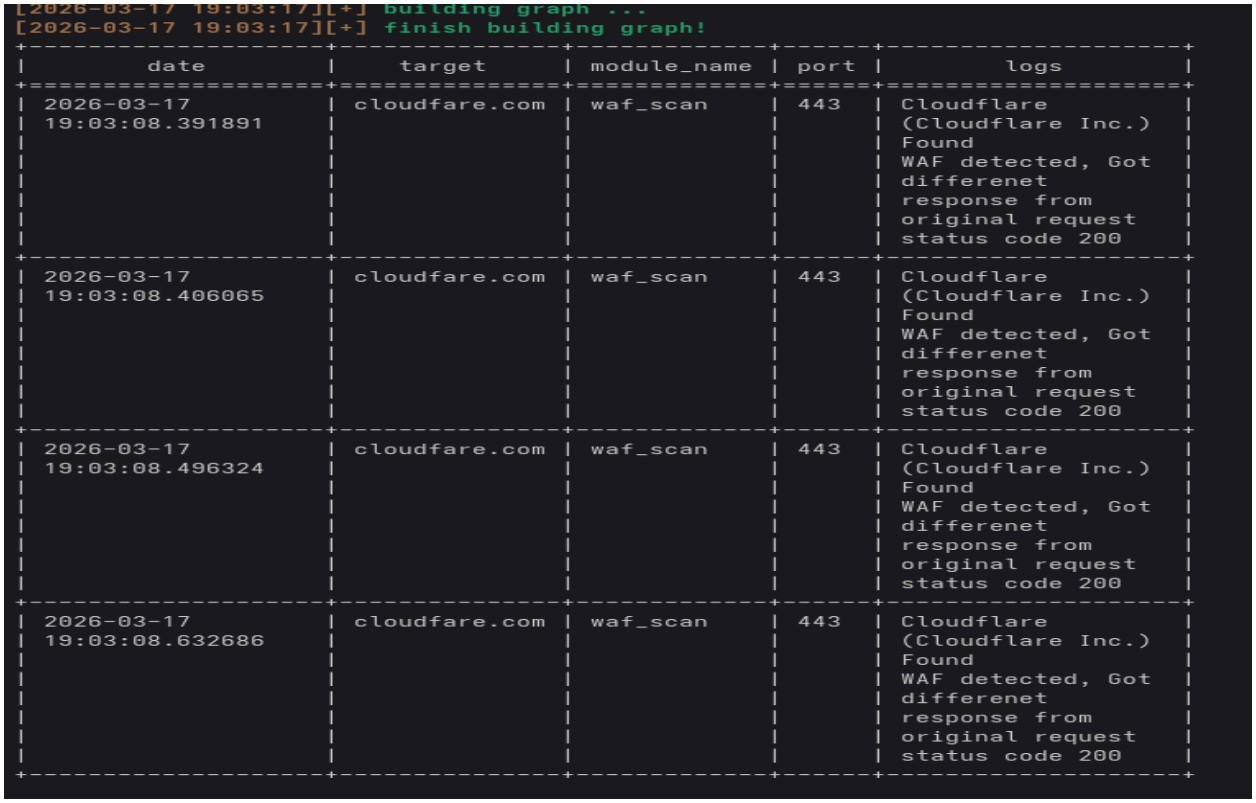
My proposal is to add a check whether any other sub-step for the same step has already logged any success or not , before executing the sub-step and in case there is a success , we can just skip the current sub-step to prevent resource wastage .

```
if "stop_at_first_success" in backup_response:
            event_name = backup_response["stop_at_first_success"]
            existing = find_temp_events(target, module_name, scan_id,
event_name)
            if existing:
                return False
```

And the logging of success is being stored in temporary database

```
if event["response"]["conditions_results"] and "stop_at_first_success"
in event.get("response", ""):
            submit_temp_logs_to_db(
                {
                    "date": datetime.now(),
                    "target": target,
                    "module_name": module_name,
                    "scan_id": scan_id,
                    "event_name":
event["response"]["stop_at_first_success"],
                    "port": event.get("ports", ""),
                    "event": event,
                    "data": response,
                }
            )
```

After the changes (Code for reference - @f0df79b) , it has drastically reduced the duplicate entries in the graph table



```
[2026-03-17 19:03:17][+] building graph ...
[2026-03-17 19:03:17][+] finish building graph!
+---------------------+----------------+-------------+------+---------------------+
|        date         |     target     | module_name | port |        logs         |
+=====================+================+=============+======+=====================+
| 2026-03-17          | cloudfare.com  | waf_scan    | 443  | Cloudflare          |
| 19:03:08.391891     |                |             |      | (Cloudflare Inc.)   |
|                     |                |             |      | Found               |
|                     |                |             |      | WAF detected, Got   |
|                     |                |             |      | differenet          |
|                     |                |             |      | response from       |
|                     |                |             |      | original request    |
|                     |                |             |      | status code 200     |
+---------------------+----------------+-------------+------+---------------------+
| 2026-03-17          | cloudfare.com  | waf_scan    | 443  | Cloudflare          |
| 19:03:08.406065     |                |             |      | (Cloudflare Inc.)   |
|                     |                |             |      | Found               |
|                     |                |             |      | WAF detected, Got   |
|                     |                |             |      | differenet          |
|                     |                |             |      | response from       |
|                     |                |             |      | original request    |
|                     |                |             |      | status code 200     |
+---------------------+----------------+-------------+------+---------------------+
| 2026-03-17          | cloudfare.com  | waf_scan    | 443  | Cloudflare          |
| 19:03:08.496324     |                |             |      | (Cloudflare Inc.)   |
|                     |                |             |      | Found               |
|                     |                |             |      | WAF detected, Got   |
|                     |                |             |      | differenet          |
|                     |                |             |      | response from       |
|                     |                |             |      | original request    |
|                     |                |             |      | status code 200     |
+---------------------+----------------+-------------+------+---------------------+
| 2026-03-17          | cloudfare.com  | waf_scan    | 443  | Cloudflare          |
| 19:03:08.632686     |                |             |      | (Cloudflare Inc.)   |
|                     |                |             |      | Found               |
|                     |                |             |      | WAF detected, Got   |
|                     |                |             |      | differenet          |
|                     |                |             |      | response from       |
|                     |                |             |      | original request    |
|                     |                |             |      | status code 200     |
+---------------------+----------------+-------------+------+---------------------+
```

Remaining Work

- I have to figure out why it still contains 4 entries and fix that , though it has reduced the entities count from about 80 to 4 (note that this is for the updated waf module with new payloads , whose PR is yet to get merged)
  (I think that duplicate entries in the graph table was the reason that many potential payloads were ignored in the original implementation of the waf module)

2. AND Work-flow for modules

- Currently, Nettacker does not support structured workflows between modules. I propose introducing an **“AND” workflow mechanism**, where the execution of dependent modules is conditional on the successful outcome of preceding modules.

  Code for reference - @f0df79b

Core Idea

The system builds a **directed graph (`flow_graph`)**, where:

- graph[module] = [list of dependencies]
- Execution is governed by **dependency satisfaction and success state**

```
        def dfs(node):
                if node in stack:
                    die_failure(_("cycle_detection_error"))
                if node in visited:
                    return
                stack.add(node)
                for parent in graph[node]:
                    dfs(parent)
                stack.remove(node)
                visited.add(node)

            for node in graph:
                dfs(node)

            options.module_flow_graph = graph
        else:
            options.module_flow_graph = None
```

Workflow Algorithm

- Graph initialization
- Iterative Scheduling loop
- Module Scheduling Conditions
  For each module, skip if
  - Running , blocked, completed
  - Block Condition: If any dependency module is yet to get executed or in running state

```
for module_name in graph:
                        if (
                            module_name in state["completed"]
                            or module_name in state["blocked"]
                            or module_name in state["running"]
                        ):
                            continue

                        dependencies = graph[module_name]

                        # dependency blocked
                        if any(dependency in state["blocked"] for
dependency in dependencies):
                            state["blocked"].add(module_name)
                            continue
```

- Execution Condition: If all the dependencies are resolved with successful detections , then launch the module

Interface to user :

- poetry run python3 nettacker.py -i "cloudfare.com" -m all --module-flow "port_scan->waf_scan,waf_scan->dir_scan,icmp_scan->wp_plugin_scan"

This is just an example of how module flow can be given in CLI using A->B format and it is being taken care that if user give circular dependencies then the error will be thrown .

```
        def dfs(node):
```

```
            if node in stack:
                die_failure(_("cycle_detection_error")) // this line
is taking care of cyclic dependencies
```

Remaining Work

- Implement “OR” workflow , so the following module work-flows can also work :
    - Execute module D and C only if either of Module A or B gave successful results
- Event driven Work-flows , e.g. launch module A if this is some event has given successful result , this will help to implement modules with dependencies on other modules

Improving Reconnaissance Scans

The following areas need improvement in Nettacker:

1. Nettacker does not provide the **versions** of the services being found on open ports.

- The regexes present in port.yaml are only to match the service and not the versions. This is not ideal because hosts might be running outdated versions of some services that might be exploitable.
- With this additional feature, Nettacker will allow users to identify vulnerable and outdated services, which is good.
- In many cases, we can get some additional information about the device type , Operating system and hardware platform as well based on the response which can be useful to identify the exploitable services and improves user experience.

2. Currently, there is no support for detecting UDP-based services. Incorporating this capability would significantly enhance the system’s

effectiveness, especially given the growing adoption of HTTP/3, which leverages QUIC over UDP instead of TCP.

The following will cover these in detail.

Problems

There are two issues now:

1. Probing using **GET / HTTP/1.1** is not ideal because HTTP services now get **incomplete requests**. (this limitation was there last year as well)

Many Services start returning **error packets** because of authentication failures, and give no useful information about themself (except the error messages which can act as characteristic for services , will be explained later in detail) . The regex therefore doesn't work.

2. Some services don't return anything when probed with arbitrary bytes

These services are usually UDP based or they are FTP services like SMB, RDP etc. that require authentication before they respond. Additionally there are protocols like FTP that leak their banners without needing probes.

These issues will be addressed by creating a different function that will adjust its probes based on the port it's probing, so in short payload-based probing is a solution for these issues .

Version Scanning and Better Probes

An end-to-end implementation of version scanning can be found over this changelog in my local fork.
This relies on a YAML file that is formed using the nmap-service-probes.txt file.
The generated YAML file will be structured as follows:

```
Format:
```

```
Excluded Ports:
  TCP: 3454,2345,3455-3453
  UDP: 3443-2453,34
  Universal: 23,53,6-345

probe:
  - name: "NULL"
    protocol: TCP/UDP
    totalwaits: xxx
    tcpwrappedms:
    rarity: 1-9
    ports: [23,24,35-654]
    sslports:
    fallbacks: ["s1", "s2"]
    probe_payload: ""
    payload:
    signatures:
      - type: "match"
        service: "http"
        regex: "..."
        Ignore_case:
        New_line_specifier:
        version:
          raw:
          product: ""
          version_template:
          info:
          hostname:
          operating_system:
          device_type:
          cpe:
            service: ""        # /a
            operating_system: ""  # /o
            hardware_platform: "" # /h
```

```
    - type: "softmatch"
      service: ""
      regex: ".."
      options: "s/i/both"
      version:
        raw:
        product: ""
        version_template:
        info:
        hostname:
        operating_system:
        device_type:
        cpe:
          service: ""         # /a
          operating_system: ""  # /o
          hardware_platform: "" # /h
```

Parser - **Gdrive link** (used to generate the YAML file)
YAML file - **Gdrive link**
Probe Loader - Gdrive link
Probes Engine - Gdrive link
This official documentation for the file format of nmap-service-probes explains the meaning behind **the flags** that are used in the YAML file.

Complete Flow

- Probes are loaded in app.py using probes_loader.py and probes.yaml
- In port.yaml , tcp_and_udp_scan method is called which then calls tcp_connect_send_and_receive to filter out the filtered ports then use ProbeEngine to probe sequentially using payload-based-probes.

```
        def tcp_and_udp_scan(self, host, port:int, timeout):
        probes_by_name = build_probes_from_yaml()
```

```
        tcp_status = tcp_connect_send_and_receive(host , port ,
timeout)
        tcp_result = None
        udp_result = None
        if tcp_status["status"] != "closed":
            engine = ProbeEngine(
                    port= port,
                    protocol="tcp",
                    host= host,
                    probes_by_name=probes_by_name,
            )
            tcp_result = engine.probe_sequentially()
            if tcp_result != None:
                return tcp_result
        udp_status = udp_scan( host , port , timeout)
        if udp_status["status"] != "closed":
            engine = ProbeEngine(
                    port= port,
                    protocol="udp",
                    host= host,
                    probes_by_name=probes_by_name,
            )
            udp_result = engine.probe_sequentially()
            if udp_result != None:
                return udp_result
```

- Probe_sequentially follows the way how nmap handles the types of responses and matches which is explained in detail in [This document](), then returns the result in case of successful service detection with following format

```
{'response': b'', 'service': 'ssl', 'ssl_flag': True, 'log':
{'version_template': '', 'product': '', 'info': '', 'hostname': '',
'operating_device': '', 'device_type': '', 'cpe_service': '', 'cpe_os':
```

```
'', 'cpe_h': '', 'cipher': ('TLS_AES_256_GCM_SHA384', 'TLSv1.3', 256),
'tcp_wrapped': False, 'peer_name': ('8.8.8.8', 443)}}
```

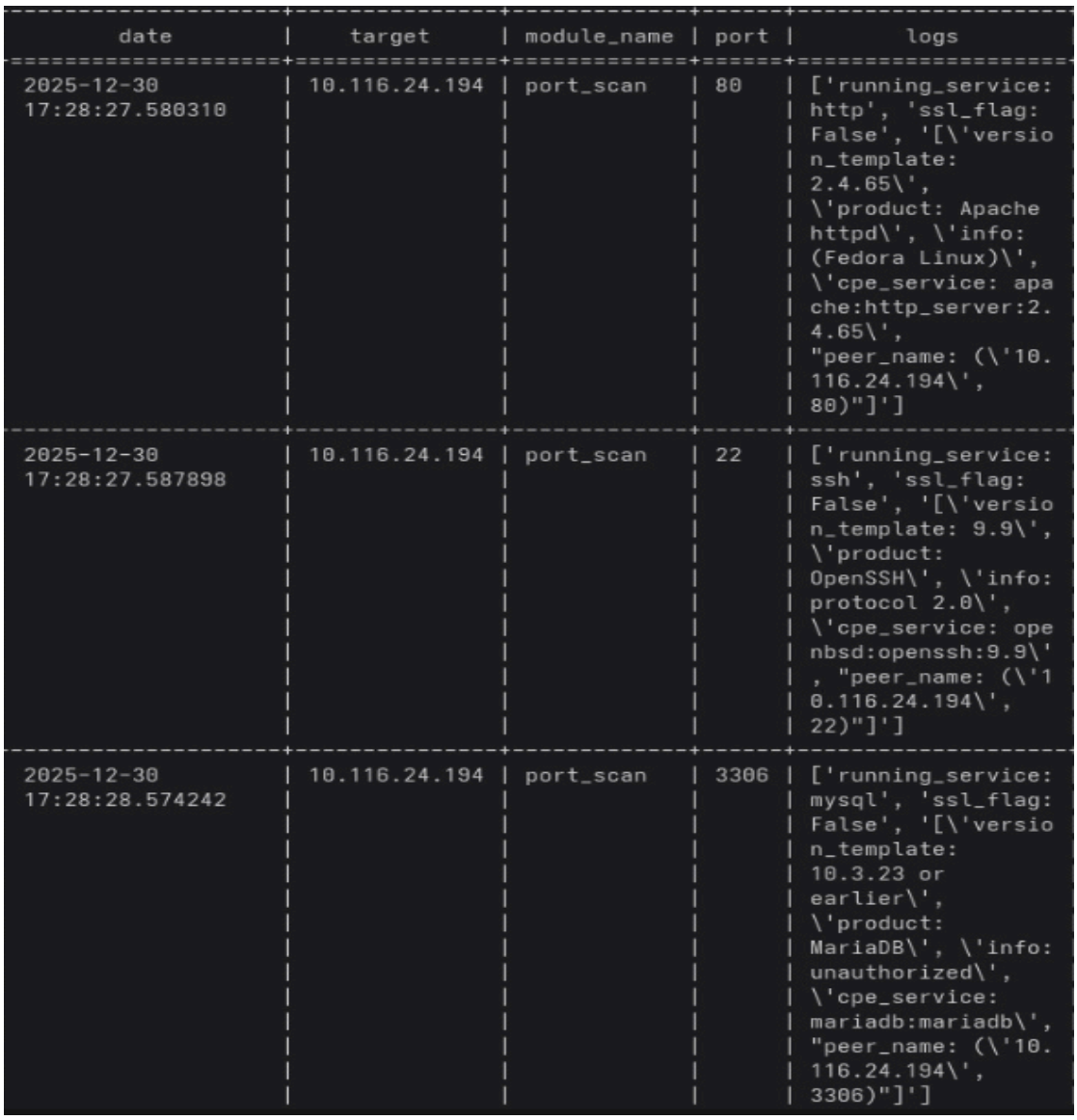
```
'service': 'http', 'ssl_flag': False, 'log': {'version_template':
'2.4.7', 'product': 'Apache httpd', 'info': '(Ubuntu)', 'hostname': '',
'operating_device': '', 'device_type': '', 'cpe_service':
'apache:http_server:2.4.7', 'cpe_os': '', 'cpe_h': '', 'cipher': None,
'tcp_wrapped': False, 'peer_name': ('45.33.32.156', 80)}}

{'response': b'SSH-2.0-OpenSSH_9.9\r\n', 'service': 'ssh', 'ssl_flag':
False, 'log': {'version_template': '9.9', 'product': 'OpenSSH', 'info':
'protocol 2.0', 'hostname': '', 'operating_device': '', 'device_type':
'', 'cpe_service': 'openbsd:openssh:9.9', 'cpe_os': '', 'cpe_h': '',
'cipher': None, 'tcp_wrapped': False, 'peer_name': ('10.215.121.198',
22)}}
```

There are 6 flags defined in the nmap docs which will be used here. The doc has been linked here.

- The probes are sent over the socket and a response is awaited. If a response is received, it is sent to the **match_regex** function along with the regex_values_dict_list.

The implementation for the match_regex function can be found in ProbeEngine in my local fork.

Results due to Payload-based Probes :



date	target	module_name	port	logs
2025-12-30 17:28:27.580310	10.116.24.194	port_scan	80	['running_service: http', 'ssl_flag: False', '[\'version_template: 2.4.65\', \'product: Apache httpd\', \'info: (Fedora Linux)\', \'cpe_service: apache:http_server:2.4.65\', "peer_name: (\'10.116.24.194\', 80)"]']
2025-12-30 17:28:27.587898	10.116.24.194	port_scan	22	['running_service: ssh', 'ssl_flag: False', '[\'version_template: 9.9\', \'product: OpenSSH\', \'info: protocol 2.0\', \'cpe_service: openbsd:openssh:9.9\', "peer_name: (\'10.116.24.194\', 22)"]']
2025-12-30 17:28:28.574242	10.116.24.194	port_scan	3306	['running_service: mysql', 'ssl_flag: False', '[\'version_template: 10.3.23 or earlier\', \'product: MariaDB\', \'info: unauthorized\', \'cpe_service: mariadb:mariadb\', "peer_name: (\'10.116.24.194\', 3306)"]']

The logs include Version along with other information like product , cpe_service (Common Platform Enumeration Service) etc. which can be useful for further Scanning of the target .

Remaining work

1. Parallelize probes across threads

The current execution model requires each thread to iterate through the entire probe list, which introduces redundant computation and may not be optimal. An alternative approach would be to assign a single probe (or a subset of probes) to each thread, thereby reducing repeated traversal of the probe list.

However, since the system already parallelizes execution across targets and ports, introducing an additional layer of parallelism at the probe level may lead to increased overhead, such as higher thread management costs and resource contention. As a result, the net performance benefit of this change is not immediately clear.

A thorough evaluation through benchmarking and profiling is required to determine whether probe-level parallelization offers a meaningful improvement or simply adds unnecessary complexity.

2. Add Service detection as an option

In the current implementation , I have replaced the old way of service detection which might be unnecessary for some users . So , I will add this service detection as an option (-sv) .

3. Add the intensity control of the version detection

Since , for each probe we have a field named “rarity” which corresponds to how infrequently this probe can be expected to return useful results. The higher the number, the more rare the probe is considered and the less likely it is to be tried against a service .
Please refer to This document for details of the rarity field .
I will have to add an option to take the intensity level as input from the users (with default value of 5) and then it will only execute the probes having rarity level lesser than the intensity level .

4. Improve the logs in final graph

The logs shown above are not visually structured and appear cluttered. I plan to improve their formatting to make them more readable, organized, and professionally presented.

Differentiating between TCP and UDP

Nettacker currently analyses only TCP ports. This is not ideal because services like DNS, NetBIOS Name Service etc. will not be detected by Nettacker. This requires probing specifically for UDP ports and logging them neatly.

The implementation of payload-based probes takes care of UDP services too .

- The probes in the YAML file are formatted to include the protocol (either TCP or UDP), so for ports that are pre-assigned to UDP by default, these UDP probes will be used.

```
- name: DNSVersionBindReq
  protocol: UDP
  totalwaits: 5000
  tcpwrappedms: 5000
  rarity: 1
  ports:
  - 53
  - 1967
  - 2967
  - 26198
  sslports: []
  fallbacks: []
  probe_string:
\0\x06\x01\0\0\x01\0\0\0\0\0\0\x07version\x04bind\0\0\x10\0\x03
  no_payload: false
  signatures:
  - type: match
    service: domain
    regex:
```

```
\x07version\x04bind\0\0\x10\0\x03\xc0\x0c\0\x10\0\x03.{7}(\d[-\w.+]*?)-Red Hat-[-\w._+]+.fc(\d+)
    Ignore_case: false
    New_line_specifier: true
    version:
      raw: p/ISC BIND/ v/$1/ i/Fedora Core $2/ o/Linux/ cpe:/a:isc:bind:$1/ cpe:/o:fedoraproject:fedora_core:$2/
      product: ISC BIND
      version_template: $1
      info: Fedora Core $2
      hostname: ''
      operating_device: Linux
      device_type: ''
      cpe:
        cpe_service: isc:bind:$1
        cpe_os: fedoraproject:fedora_core:$2
        cpe_h: ''

(And more signatures, terminated here for better readability )
```

A brief summary for the udp scanning can be found below:

- If the probes based detections fail for TCP protocol for a particular port and target , then UDP protocol is used to detect service . I am using this sequence of operations because whatever information UDP probes can provide are also provided by TCP probes (so its better to first run TCP scans , so that we can get result if TCP protocol is supported by the target on the particular port) and then execute the UDP - probes.

Remaining Work

The remaining work is mostly the same as those for service logging. It's referenced here.

External API Key Management Integration

Issues

- Currently there is no support to manage and use external API keys like shodan, VirusTotal etc.
- Once this support has been added, there is no way of extracting the relevant info from the valuable data we get from the APIs such as the following data excerpt from shodan API endpoint :


```
{'reason': 'OK', 'url':
'https://api.shodan.io/shodan/host/8.8.8.8?key=Qn21sZWFfOPbdoKLomsMFamFf
C2WJHWi', 'status_code': '200', 'content': '{"city": "Mountain View",
"region_code": "CA", "os": null, "tags": [], "ip": 134744072, "isp":
"Google LLC", "area_code": null, "longitude": -122.11746, "last_update":
"2026-03-20T13:59:54.514200", "ports": [443, 53], "latitude": 38.00881,
"hostnames": ["dns.google"], "country_code": "US", "country_name":
"United States", "domains": ["dns.google"], "org": "Google LLC", "data":
[{"hash": -553166942, "opts": {}, "timestamp":
"2026-03-20T10:22:05.978399", "isp": "Google LLC", "data":
"\\nRecursion: enabled", "_shodan": {"id":
"19ec229b-9b57-4db5-b3e2-4b173b03701c", "region": "na", "options": {},
"module": "dns-tcp", "crawler":
"a4815a8bfadb2b966b4697ac569130d1cfb605d4"}, "port": 53, "hostnames":
["dns.google"], "location": {"city": "Mountain View", "region_code":
"CA", "area_code": null, "longitude": -122.11746, "country_name":
"United States", "country_code": "US", "latitude": 38.00881}, "dns":
{"resolver_hostname": null, "recursive": true, "resolver_id": null,
"software": null}, "ip": 134744072, "domains": ["dns.google"], "org":
"Google LLC", "os": null, "asn": "AS15169", "transport": "tcp",
```

```
"ip_str": "8.8.8.8"}, {"hash": -553166942, "opts": {"raw":
"34ef818200010000000000002696406736572766572000010003"}, "timestamp":
"2026-03-20T13:59:54.514200", "isp": "Google LLC", "data":
"\\nRecursion: enabled", "_shodan": {"region": "", "module": "dns-udp",
"ptr": true, "options": {}, "id":
"690f8101-a330-4224-9607-c9652521d106", "crawler":
And so on
```


To address this, a basic external API key management system has been introduced. Users can now securely store API keys (such as Shodan) using API endpoints, instead of hardcoding them inside modules. These keys are centrally managed and can be reused across different modules.

During scan execution, the required API key is dynamically injected into the module inputs, allowing the module to make authenticated API calls as

```
        self.module_inputs = options.__dict__
        self.module_inputs["target"] = target
        for service, value in api_key_manager.keys.items():
            self.module_inputs[f"{service}_key"] = value
```

A working integration with the Shodan API has been implemented, where the module successfully fetches data using the stored API key.

date	target	module_name	port	logs	json_event
2026-03-19 19:55:18.004018	8.8.8.8	shodan_scan		Shodan data found for 8.8.8.8	▸ {...}

Remaining Work

There are **three issues** that I will need to fix to make this implementation robust:

1. I have to add UI support to let user add the API key through that , currently I have done the following to add the API key :

```
curl -k -POST "https://127.0.0.1:5000/api/external-api-keys?key=xxxx" \
  -H "Content-Type: application/json" \
  --data-raw '{
    "service": "shodan",
    "value": "xxxxx"
  }'

I have removed the Api keys from the command
```

2. I have to make modifications to allow in general all the APIs along with implementation of other common APIs like Censys and VirusTotal.
3. A centralised parsing system to parse the response we get from such API calls , we can't use current matchers approach to extract relevant info or even to log the info we get from the API calls

Extractors integration with modules

ISSUE

- Currently, Nettacker only supports matchers to verify a successful detection and doesn't have a feature to extract useful information which could improve the user experience.

The integration of extractors with modules could potentially lead to integration of more robust modules in nettacker where we can extract useful information along with detection of vulnerability.

I would like to propose the idea of integrating the extractors with http library based modules and I have done basic implementation of such extractors in Code for reference and used the shodan module to verify if it's working as expected.

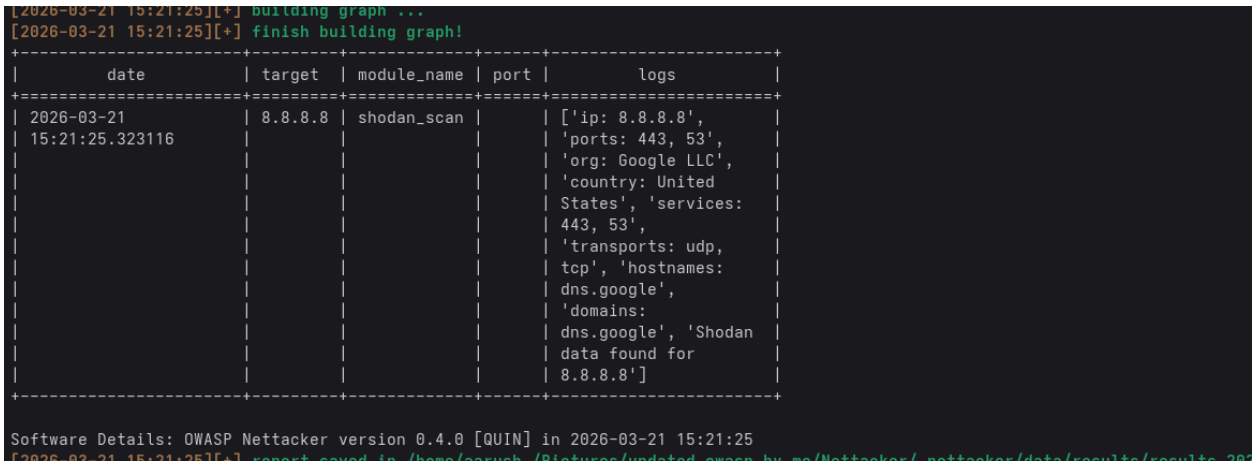
I have utilized the “jq” Python library to parse JSON responses, as it offers a more accurate and efficient approach compared to manual parsing.

For the output we get from shodan API call , I have used the following extractors on that :

```
extractors:
  ip:
    type: json
    name: ip
    json:
      - .ip_str
  ports:
    type: json
    name: ports
    json:
      - .ports[]
  org:
    type: json
    name: org
    json:
      - .org
  country:
    type: json
    name: country
    json:
      - .country_name

  services:
    type: json
    name: services
    json:
      - .data[]?.port

  transports:
    type: json
    name: transports
    json:
      - .data[]?.transport
```

```
  hostnames:
    type: json
    name: hostnames
    json:
      - .hostnames[]

  domains:
    type: json
    name: domains
    json:
      - .domains[]
```

After running the shodan_scan with target as 8.8.8.8, the result looks as follows:



Remaining Work

1) Implement XPath Extractor to extract data from HTML/XML responses.
2) Key-Value extractor to extract data from HTTP headers and other fields, currently it only supports extraction from content.
3) A DSL extractor to process response data (e.g., string operations, length, encoding), making extraction more flexible, by extending the DSL engine already in the PR#1153 by rxerium.

Web Server Improvements

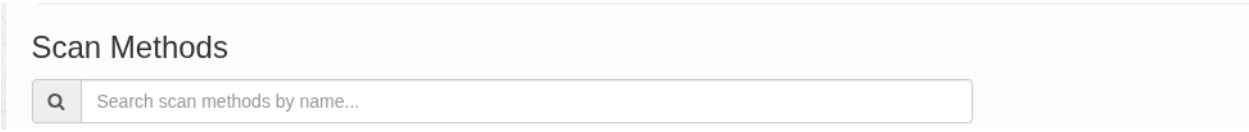
There are three things that I want to implement here that a user would benefit from:

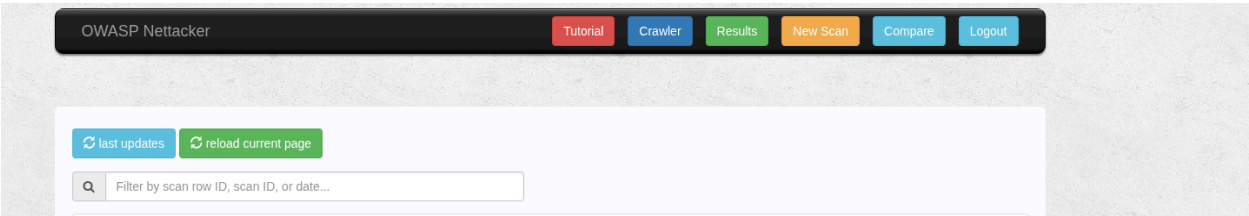
1. Add **search functionality** to Profiles, Scan Methods, and Results (filter by scan ID, date, or index) for faster navigation.
2. Improve UI by organizing Profiles and Scan Methods into **scrollable/sliding sections** for better usability with large datasets.
3. Introduce a **dashboard view** to visualize scan results and provide a more structured overview of activity.

Search functionality

This mainly requires directly modifying main.js and index.html

The basic case has been implemented over in this changelog
The field will be placed like this:

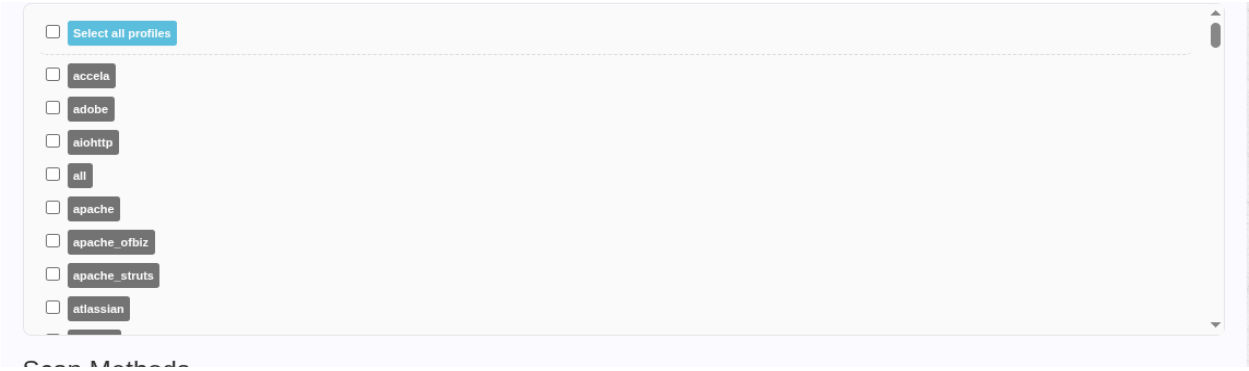


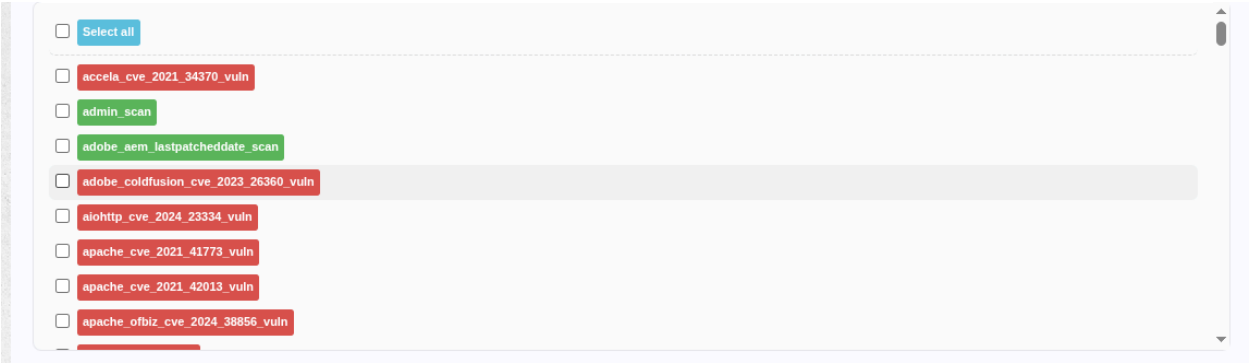


Sliding sections

This mainly involves modifying the layout and styling in index.html and ui_upgrade.css, along with minor adjustments in main.js for dynamic rendering.

The updated layout will present modules in a cleaner and more readable format, making it easier to navigate and select items as :





Remaining Work

Remaining work includes developing a dashboard to visualize scan results and improve the overall understanding and usability of the output.

Improving Test coverage

I raised PRs #1319 (merged) to familiarize myself with the general test structure.

The timeline will explain how and when I plan to write test cases. I hope to finish writing major test files before the coding phase of GSoC, so that I can focus on writing tests for my ideas and new files .

Remaining Work

To develop a dashboard (using Streamlit) for visualizing scan results and providing a structured, real-time overview of scanning activity.

Timeline

Application Review Period

March 31st - April 30th

- I will focus on **preparing my code** for the coding period to avoid potential delays.
    - I will start to make my implementation of **version scanning** more robust and start ironing out issues. This will allow me more time to work on it and get it right.
    - I will push a PR to support new module workflows.
    - Additionally, I will start **implementing tests** to improve test coverage as much as possible.

Community Bonding Period

May 1st - May 24th

- My end-of-semester exams will start from May 2nd and continue until May 11th. I will need a week leading up to my exams in order to prepare.

- I plan to use the Community Bonding Period in its true sense and learn more about OWASP. I have a few more things I want to try and explore under owasp, especially I am interested in exploring the ZAP project under OWASP.

- I will push 2-3 new CISA KEV vulnerability modules during this period.

- I will also continue to spend this time on my project ideas and convert the pseudocodes and ideas I provided above to fully fledged ones.

  This will take up the majority of this period. Specifically, I will use this time to complete the test coverage (if not able to complete it in the Application review period) , and I hope to finish the majority of them, and fix issues with my version scanning approach.

GSoC Period

I plan on dividing my time in 4 distinct phases of 3 weeks each. The following neatly describes the time I plan on taking to implement each idea properly in each phase.

I have included my **priority** preference with the same, and I have kept the high priority task based on what requires the **most work and will be beneficial** to Nettacker.

Note that I will be working on extending the coverage of CISA KEV vulnerability modules in parallel for the whole duration of GSoC and I am aiming to contribute at least one module every week, so I am not adding that task in the following phase by phase timeline.

Phase 1 (week 0 - 3)

May 25th - June 15th

Medium priority

- I will continue working on the PR I raise during the Application review Period for the **New module workflows** and aim to finish it in the first 2 weeks.
- I will push 5 PRs for **5 test files** in parallel . The test cases will be for the files :
  - module.py
  - messages.py
  - core/lib/ssh.py
  - core/lib/smtp.py

    - core/lib/telnet.py
- I will also start implementing **version logging** and will continue to complete it in phase2 .

Phase 2 (week 3 - 6)

June 15th - July 6th

High priority

- I will continue working on improving **version logging**. I hope to finish this by the 2nd week of this phase with end to end integration .
- In parallel, I will push 2 PRs for 2 test files :
    a. app.py
    b. base.py
- I chose these because they will be modified by my implementation of version scanning, and hence could be covered after version scanning implementation is done.
- Assuming that DSL will get integrated by the time, I will update the test file test_yaml_schema_and_regex.py to cover the new modules with DSL usage.
- I will focus on porting the CISA KEV modules that require the DSL and have not yet been integrated into Nettacker and start the next phase independently.
- Additionally, after making these changes, I will dedicate extra time to include tests for them.

Phase 3 (week 6 - 9)

July 6th - July 27th

Medium priority

- Here, I will focus on integration of external API keys (shodan, Censys, VirusTotal and SecurityTrails)
- In the first week , I will finish the integration of the shodan API and Censys along with the implementation of its parser .
- In parallel, I will create 3 test cases for the following 3 files:
    a. core/lib/ftp.py
    b. api/engine.py
    c. template.py

- In case my 15-day NUS visit (detailed in the Time Commitment section) affects my availability, I will adjust the timeline by moving the creation of the three test files to the last week of the next phase.
- By the end of the 2nd week , I am aiming to complete the integration of SecurityTrails and VirusTotal.
- During the last week, I will start preparing the integration of the extractor.

Phase 4 (week 9 - 12)

July 27th - August 18th

Medium priority

- I aim to complete the extractors integration by the end of the first week .
- In the 2nd week, I will start implementing a dashboard using Streamlit to visualize scan results.

I would like to keep the **final week as a buffer** to work on any unfinished ideas. In the case where there are none, I will spend this time writing documentation for all the changes I have made.

I usually write draft notes in parallel to whatever I do, so I will refine them for a more professional style.

Post GSoC Period

- I want to develop Nettacker into a better overall tool for industrial recognition, than it was when I started. After my tenure in GSoC I will try to guide my fellow contributors into developing this further by providing new suggestions and reviewing their code.
- In case I have any pending PRs related to GSoC, I will ensure that I finish them first.
- I will update the README and documentation (if the maintainers need me to) in order to document all the new features and changes made, if I cannot do this during the final week.

OWASP Nettacker was my introduction to open source, so it holds special significance for me. I intend to continue contributing to the project in any way I can, while also integrating new cybersecurity concepts and techniques that I learn along the way.

I also have a few new ideas that I wasn't able to fully articulate earlier. I plan to discuss their feasibility with the mentors and explore whether we can work on them before moving on to any major code refactoring.

I will strive to share my knowledge of this tool with my peers and juniors, encouraging them to explore and contribute to the project. I also hope to return next year as a mentor, provided I gain sufficient experience and understanding.

Time commitments

- I will be traveling to Singapore, specifically to the National University of Singapore (NUS), for a 15-day visit as part of an **ongoing research project** in the field of **Computer Networks**. This is a pre-planned commitment, and I will ensure that it does not result in any delays in fulfilling my responsibilities.
- My work will majorly be during the morning hours from **6am to 5pm** and I will be free to pursue my **GSoC goals after that period**.

My timeline also accounts for this delay, and I have ensured that my work will not be hindered and leftover work is completed in the end week of the GSoC phase.

I will be free during the month of August and can easily compensate for any potential backlogs during August.

- Other than that, I have no other commitments, and in the weeks leading up to August, I can easily dedicate upwards of 30 hours a week to ensure consistent progress.

If I am caught up with any work (personal or miscellaneous) and might miss a deadline or a weekly meeting, I would inform my mentor way beforehand and would also indicate the date spans where I would put in more time and get the job done.

Acknowledgements

As I finish my proposal, I want to thank @securestep9 for guiding me through my first PRs and opening up the world of open-source for me. I also want to thank @pUrGe12 for introducing the open-source world to me and helping a lot in the initial phase with my silly doubts, and helping me with my proposal and giving insanely good feedback.